The Dash Against Time: The Shortest Path to Riches

Evan Liu

School of Computing, Binghamton University

CS 301 Ethical, Social, & Global Issues in Computing

Professor Weinschenk

November 8, 2026

The Dash Against Time: The Shortest Path to Riches

After another DoorDash delivery, Kelly peeks at her gas gauge. Her phone then buzzes with a new order, a moment when her route can affect her earnings. She is offered $8 for five miles, which does not meet the $2-per-mile rate gig drivers use to decide whether to accept an order. She decides to decline the order because gas expenses would eat into her earnings, but with an efficient route, the order could have been brought within the rate. Navigation software uses algorithms to find the quickest or shortest route. For example, Dijkstra's algorithm looks at all possible routes based on total cost, whereas A* uses estimates to find the remaining distance to the goal. By exploring fewer nodes than Dijkstra's algorithm, A* reduces search time (Elshaer et al., 2025) and enables navigation software to reroute gig drivers more efficiently. The paper reviews research on both algorithms' performance in transportation, as well as studies on the social impact of faster route calculations on gig drivers. First, the paper goes over how Dijkstra finds routes, then how A* narrows the search, and finally what the difference means for drivers' lives. Understanding these differences starts with how Dijkstra calculates the shortest path and the limitations that come with the algorithm.

Technical Background of Alternative Algorithm

Navigation software relies on pathfinding to determine the least expensive way to travel between two points on a road system. The first step in creating an algorithm is to model the road system.

as a graph, where nodes represent intersections and edges represent road segments (Altcademy Team, 2023), and assign values to each edge based on travel time. One of the earliest search algorithms introduced in 1959 is Dijkstra's. The algorithm is a predecessor to other search methods like A*. Dijkstra's algorithm assigns a distance value to every node: zero for the starting node and infinity for the rest. The algorithm uses a priority queue to pick the unvisited node with the smallest distance. After popping a node from the queue, the algorithm checks each neighbor. If the path through the current node costs less than the neighbor's recorded distance, the algorithm updates that distance. The search works because once the shortest distance to a node is found, all other nodes must be farther away from the starting node, so the distance cannot decrease. Figure 1 illustrates the process. Table 1 traces each step for a trip from node A to node D. After removing A, the algorithm records B at 4 minutes and C at 2 minutes. Next, the algorithm removes C, finds that B is cheaper through C (3 minutes), and records D at 10 minutes. Finally, the algorithm removes B and finds that D is cheaper through B (8 minutes). When D is removed from the queue, the search ends. The algorithm picks the route A to C to B to D. The time complexity of Dijkstra's depends on the priority queue's implementation. For example, a binary heap takes O((V + E) log V) time, as shown in Equation 1, where V is the number of nodes and E is the number of edges.

$$O((V + E) \log V) \tag{1}$$

Dijkstra's algorithm has one main disadvantage: the search considers only the cost already traveled and has no information about the goal. As a result, the search expands outward in every direction (Altcademy Team, 2023), exploring many nodes that lead away from the goal before

finally reaching the destination. A* is designed to solve unguided searches. For gig drivers who need a new route immediately after a delay or a missed turn, extra search time means a slower response from the navigation app.

Figure 1

Example Road Network with Travel Times in Minutes

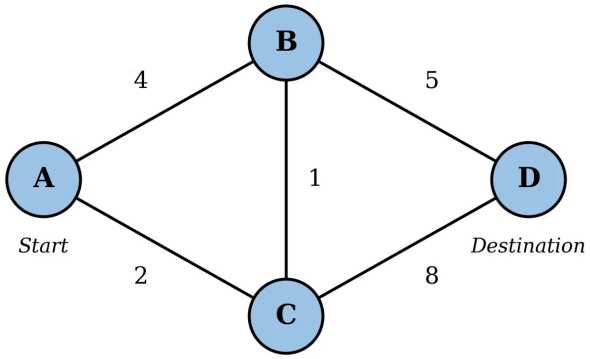


Table 1

Dijkstra's Algorithm Trace From Node A to Node D

Step	Node Removed (Cost)	B	C	D
Start	None	Infinity	Infinity	Infinity
1	A(0)	4	2	Infinity
2	C(2)	3	2	10
3	B(3)	3	2	8

4	D(8)	3	2	8

Technical Details

A* solves these problems by giving the search direction. Hart et al. (1968) introduced A* as a way to incorporate problem knowledge into graph search while still guaranteeing the cheapest path. Instead of ranking nodes only by the cost already traveled, A* ranks each node n* by the estimated total cost of a route that passes through a node, as shown in Equation 2.

$$f(n) = g(n) + h(n) \tag{2}$$

In Equation 2, g(n) is the same value as Dijkstrasa's algorithm records: the cost of the best known path from the starting node to n. The new term, h(n), is a heuristic, which is an estimate of the remaining cost from n to the goal. The priority queue now removes the node with the smallest f(n) instead of the smallest g(n). Every other step stays the same as in Dijkstra's algorithm. The algorithm checks each neighbor, updates the neighbor's cost if a cheaper path is found, and stops when the goal is removed from the queue. If h(n) is zero for every node, A* behaves exactly like Dijkstra's algorithm (Russel & Norvig, 2021). A* is therefore not a replacement for Dijkstra's algorithm but an extension of the algorithm.

Choosing the Heuristic

The performance of A* depends on the heuristic. On a road map, a common heuristic is the straight-line distance from a node to the goal. Because navigation software weighs edges by travel time rather than distance, the straight-line distance is divided by the highest speed allowed.

anywhere on the network, as shown in Equation 3, where d is the straight-line distance and v*max is the maximum speed

$$h(n) = \frac{d(n, \text{goal})}{v_{\max}} \tag{3}$$

Since no car can reach the goal faster than traveling in a straight line at top speed, the estimate never overestimates the actual travel time, making the heuristic admissible. Hart et al. (1968) proved that A* with an admissible heuristic always returns the cheapest path, which is the same answer Dijkstra's algorithm gives. Russell and Norvig (2021) describe a stronger property called consistency, shown in Equation 4, where c(n, n ') is the cost of the edge from n to its neighbor n '.

$$h(n) \leq c(n, n') + h(n') \tag{4}$$

Equation 4 means that the estimate can never drop by more than the cost of the edge just traveled. With a consistent heuristic, once a node leaves the queue, its cost is final, which is the same property that makes Dijkstra's algorithm correct. The trade-off is that an admissible heuristic must be cautious. A heuristic that guesses too high can make the search faster, but the search may then miss the cheapest route.

Applying A* to the Example Network

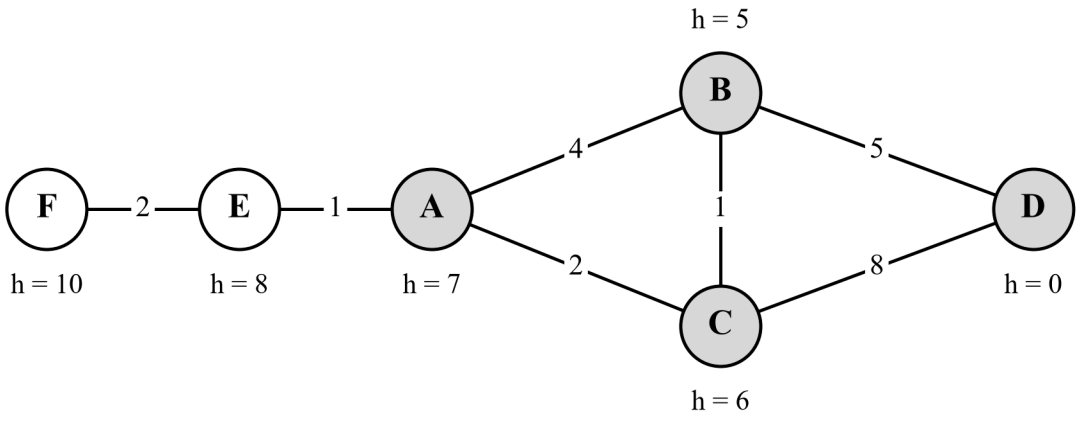
Figure 2 adds two nodes, E and F, to the network from Figure 1. These nodes sit on a road that leads away from D. Each node is labeled with a heuristic value in minutes. The values are admissible because none is larger than the true remaining time. For example, h(C) is 6, and

the fastest real path from C to D, through B, also takes 6 minutes. Table 2 traces A on the network, where each value is the node's f value, or g + h*.

In step 1, A removes A(0 + 7 = 7) and records C at 2 + 6 = 8, B at 4 + 5 = 9, and E at 1 + 8 = 9. In step 2, the algorithm removes C, finds that B is cheaper through C(3+ 5 = 8), and records D at 10 + 0 = 10. In step 3, the algorithm removes D, and the search ends. The route is A to C to B to D at 8 minutes, the same route Dijkstra's algorithm found in Table 1. Dijkstra's algorithm, however, would remove E at 1 minute before C at 2 minutes, and later remove F at 3 minutes, before reaching D, removing six nodes. A* removes only four and never discovers F. The heuristic tells the search that E and F lead away from the goal.

Figure 2

Example Road Network With Heuristic Estimates in Minutes



Note. Edge labels are travel times. Shaded nodes are the nodes A* removes from the queue on a trip from A to D. Nodes A, B, C, and D, and their edges are the same as in Figure 1.

Table 2

A* Search Trace From Node A to Node D (f = g + h)

Step	Node Removed (f)	B	C	D	E
Start	None	Infinity	Infinity	Infinity	Infinity
1	A (7)	9	8	Infinity	9
2	C (8)	8	8	10	9
3	B (8)	8	8	8	9
4	D (8)	8	8	8	9

Note. E is recorded but never removed, and F is never reached. Dijkstra's algorithm on the same network removes A, E, C, B, F, and then D.

Evidence From Expert Studies

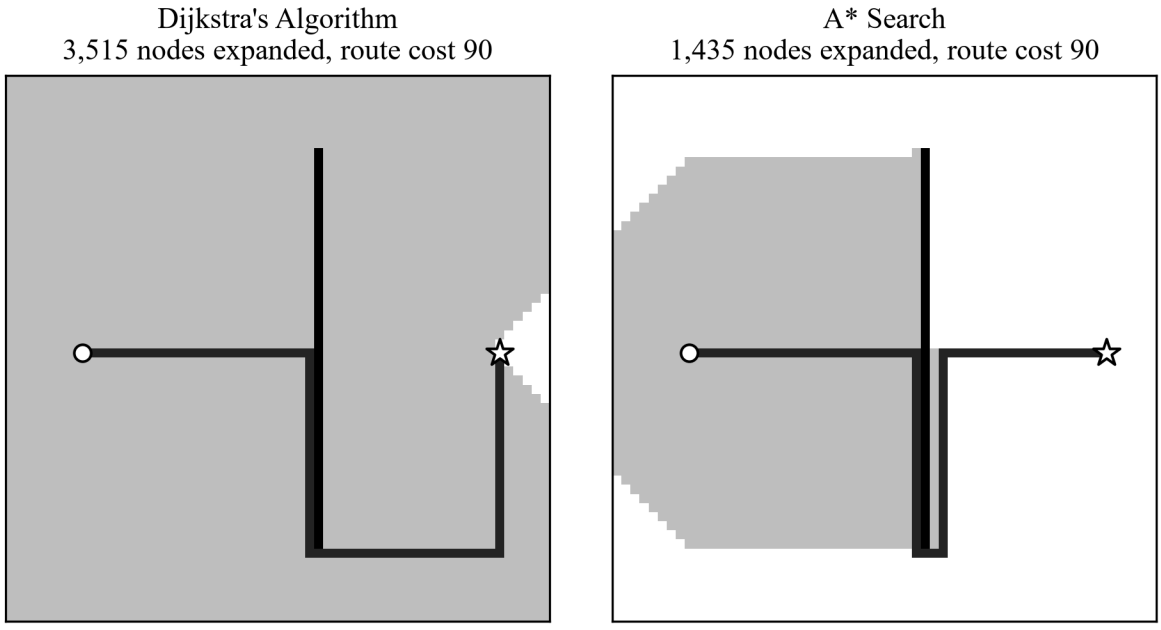
One small network shows how A* works, but it does not prove that A* is faster in general. Looking at what researchers found across many different networks shows whether the pattern holds. Elshaer et al. (2025) compared both algorithms for route planning in transportation and found that A* explored fewer nodes and reduced search time. Muhamad Azhar and Kamardan (2026) turned Google Maps data for Johor Bahru, Malaysia, into a weighted graph of intersections and roads and found that A* reached the destination in fewer iterations than Dijkstra's algorithm, while both returned the shortest route. Goldberg and Harrelson (2005), working at Microsoft Research, tested A* on road networks for driving directions. They found that plain straight-line estimates leave much of the possible speedup unused, and they introduced landmarks, which are a few chosen nodes whose distances to every other node are

computed ahead of time. Using the triangle inequality on those stored distances gave tiger estimates that were admissible, and their method beat A* with straight-line bounds by a wide margin on road networks.

To test this pattern on a larger graph, a grid simulation was run for this paper. Figure 3 shows a 60 by 60 grid where each move costs 1 and a long barrier, a river with crossings only at the ends, blocks the direct path. Dijkstra's algorithm expanded 3515 nodes, while A*, using the walking distance on the grid as its heuristic, expanded 1435 nodes, about 59% fewer. Both returned a route with a cost of 90. The barrier also shows a limitation of A*. The heuristic points straight at the goal, so A* still searches the whole area in front of the barrier before finding the detour around it.

Figure 3

Nodes Expanded by Dijkstra's Algorithm and A* on the Same Grid



Note. Gray cells were expanded; the black lines are the barrier, and the dark lines are the route found. The circle marks the start, and the star marks the goal.

Figure 4

Code used to run Figure 3

```
def search(use_astar=False):
    frontier = [(0, START)]
    distances = {START: 0}
    visited = set()

    while frontier:
        _, current = heapq.heappop(frontier)

        if current in visited:
            continue

        visited.add(current)

        if current == GOAL:
            break

        for neighbor in get_neighbors(current):
            new_cost = distances[current] + 1

            if new_cost < distances.get(neighbor, float("inf")):
                distances[neighbor] = new_cost

                h = heuristic(neighbor, GOAL) if use_astar else 0
                priority = new_cost + h

                heapq.heappush(frontier, (priority, neighbor))
```

Together, these findings lead to a general conclusion. Across a hand trace, a simulation, a real city map, and large road networks, A* returns the same optimal route as Dijkstra's algorithm while searching far fewer nodes. The amount saved depends on how close the heuristic is to the real cost. In the worst case, for example, a heuristic of zero, A* has the same time complexity as Dijkstra's algorithm, O((V + E) log V) with a binary heap. In practice, a good heuristic shrinks the part of the graph that is searched, which is why A* allows navigation software to reroute a driver quickly. The next section looks at what that speed means for gig drivers.

Social Impact

The difference between A* and Dijkstra's algorithm returning the same route is not where a driver goes but how fast and how cheaply the app decides. Starting from a few general facts

about gig work and navigation, it is possible to reason down to what that difference means for a single driver like Kelly.

Faster Rerouting Protects Thin Margins

Gig drivers are paid per order, while their costs grow with every mile and minute. Jacobs et al. (2024) studied earnings data from delivery drivers in five metro areas and found that, after expenses like gas and vehicle wear and before tips, typical delivery drivers outside California gained only $0.48 per hour, while those in California gained $5.93. With margins that thin, a few extra miles can decide whether an order is worth taking. Navigation is calculated not only once, but the app also recalculates the route after a missed turn, traffic jam, or new order added to the trip, and while it is calculating, the driver keeps moving along the old route. For these reasons, A* reduces the distance a driver travels on an outdated route and helps protect the driver's earnings per mile. Kelly's $8 order pays $1.60 per mile over 5 miles but reaches the $2 per mile rule at 4 miles. A faster search alone will save a full mile on one trip. The point is that the gap between accepting and declining is small, so every wasted fraction of a mile pushes an order closer to the wrong side of the line, and those fractions add up over dozens of orders a week.

References

Altcademy Team. (2023, June 15). Compute the shortest path between two nodes (solved). *Altcademy Blog*. https://www.altcademy.com/blog/compute-the-shortest-path-between-two-nodes-solved/

Elshaer, A. M., Elmanfaloty, R. A., Abou-Bakr, E., Elrakaiby, M., & Saada, K. (2025). Exploring algorithmic efficiency of A-Star and Dijkstra for optimal route planning in green transportation. *International Journal of Intelligent Transportation Systems Research*, *23*(2), 1097–1107. https://link.springer.com/article/10.1007/s13177-025-00503-x